אלגוריתמים 2 - הרצאה 1

13 באוקטובר 2026

גיא יער־און

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

1 תכנות לינארי (LP)

1.1 רקע

בהרצאה הקרובה נעסוק בפרדיגמה של תכנות לינארי. נלמד לנסח בעיות מעשיות באמצעות מודל מתמטי לינארי לחלוטין, נלמד 3 צורות לייצוג מתמטי של תכנות לינארי ונלמד זוג אלגוריתמים:

- Simplex algorithm: אלגוריתם פרקטי ונפוץ לפתרון כללי של בעיות תכנות לינארי.
- Ellipsoid algorithm: אלגוריתם בעל סיבוכיות פולינומיאלית אשר מבטיח פתרון יעיל, תאורטית.

בסיום, נקשר כל בעיית LP לבעיה מקבילה לה - בעיה דואלית, אשר תספק עבורנו ליצירת חסמים וכו'.

דוגמה 1. נתבונן בדוגמה ראשונה, פשוטה למדי, לבעיית מקסימיזציה תחת מגבלת משאבים.
נניח חברה שמייצרת שולחנות וכיסאות.
שולחן (x), דורש 4 שעות נגרות ו2 שעות גימור. נניח כי הרווח מכל שולחן הינו 70$.
כיסא (y), דורש 3 שעות נגרות ושעה 1 של גימור. נניח כי הרווח מכל כיסא הוא 50$.
ברשות המפעל: 240 שעות נגרות לכל היותר, 100 שעות גימור.
השאלה שלפנינו: כמה שולחנות וכמה כסאות - עלינו לייצר על מנת להביא את סך הרווח למקסימום תחת מגבלת שעות העבודה הזמינות?
נקבל, אם כן, את בעיית האופטימיזציה הבאה:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y\le 240\\ 2x+y\le 100\\ x,y\ge 0\end{cases}$$

1.2 צורות הצגה של LP

1.2.1 הצורה הכללית (Generl form)

למעשה, זוהי הצורה הטבעית והגמישה ביותר שבה הבעיה מוגדרת מלכתכילה.

- פונקציית המטרה יכולה להיות מסוג מקסימום או מינימום.
- האילוצים יכולים להיות כל אחד מהבאים: $=,\le,\ge,>,<$ והמשתנים יכולים להיות חופשיים. כלומר, ללא אילוצי סימן (יכולים להיות חיוביים, שליליים או אפס)

1.2.2 הצורה הסטנדרטית (Standard form)

צורה אחידה ונוחה לניתוח מתמטי ולחלק מן האלגוריתמים שנציג בהמשך.

- פונקציית המטרה חייבת להיות מסוג MAX (כלומר, אם הבעיה המקורית הייתה מזעור - min, אזי ממירים אותה למקסום ע"י הכפלת פונקציית המטרה ב-1).
- אילוצים: כל אחד מהאילוצים מהצורה $\le$ בלבד. וכן, כל המשתנים צריכים להיות אי שליליים.

דרך זו ניתן לייצג באופן מתמטי בשני אופנים, שקולים:

הגדרה 2. ייצוג הצורה הסטנדרטית מוגדר להיות מהצורה הבאה:

$$\max \sum_{i=1}^{n} c_i x_i \text{ s.t } \sum_{i=1}^{n} a_{1,i} x_i \leq b_1, ..., \sum_{i=1}^{n} a_{m,i} x_i \leq b_m$$

ובאופן שקול:

$$\max c^T x \text{ s.t } Ax \leq b$$

באשר, $c \in \mathbb{R}^n$ הוא וקטור המקדמים של פונקציית המטרה מגודל n, וx הוא וקטור המשתנים. $A \in \mathbb{R}^{m \times n}$ היא מטריצת האילוצים ־ שורה לכל אילוץ, עמודה לכל משתנה. $b \in \mathbb{R}^m$ הוא וקטור מהצורה $(b_1, ..., b_m)$. וכן, $\forall i : x_i \geq 0$.

1.2.3 משתני עזר (Slack form)

בצורה זו משתמשים במשוואת שוויון, היא הצורה הדרושה ישירות על מנת להריץ את אלגוריתם הסימפלקס.

- פונקציית המטרה מסוג MAX בלבד.
- כל האילוצים הם משוואות שוויון, מגיעים לכך ע"י הוספת משתני עזר המייצגים את ההפרש בין האגפים. כמו כן, כל המשתנים חייבים להיות אי שליליים.

אם כן, כיצד נהפוך מערכת אילוצי אי שוויון לאילוצי שוויון? על מנת להפוך אילוץ מהצורה $\sum_{i=1}^{n} a_i x_i \leq b_1$ לשוויון, מוסיפים משתנה אי שלילי s_1 שיקרא משתנה slack. משתנה זה מייצג את המקום הפנוי עד להגעה לגבול. פונקציית המטרה נותרת כשהייתה, ומקבלים:

$$\max \sum_{i=1}^{n} c_i x_i \text{ s.t } \sum_{i=1}^{n} a_{1,i} x_i + s_1 = b_1, ..., \sum_{i=1}^{n} a_{m,i} x_i + s_m = b_m \text{ s.t } \forall i \in [n], j \in [m] : x_i, s_j \geq 0$$

משפט 3. שלושת הדרכים לייצוג בעיית תכנות ליניארי שקולות.

1.3 Converting Unrestricted Variables (המרת משתנים חופשיים)

טכניקה נוספת להתמודד עם בעיה חשובה: כיצד נתמודד עם משתנה שאין עליו אילוץ אי־שליליות? הרי, לפי הגישה עליה הרחבנו קודם המשתנים חייבים להיות אי־שליליים. נצטרך להעביר את המשתנה לצורה שבה $z \geq 0$. לשם כך הרעיון המרכזי הוא פשוט: כל מספר ממשי ניתן להציג כהפרש של זוג מספרים אי שליליים. לכן, בהינתן משתנה חופשי z, ניתן לפצלו באופן הבא:

$$z = z_+ - z_- \text{ s.t } z_+, z_- \geq 0$$

דוגמה 4. נתבונן בבעיית האילוצים הבאה:

$$\max \ (5x + 3y) \text{ s.t } \begin{cases} x + 3y \leq 10 \\ 2x + y \leq 15 \\ x \geq 0 \end{cases}$$

על המשתנה y אין הגבלה לפיה $y \geq 0$. אך, כפי שלמדנו כל המשתנים חייבים בשיטה זו להיות אי שליליים. לכן, נוכל לבצע

המרה ולקבל באופן שקול:

$$\max\ (5x+3y_+-3y_-)\ \text{s.t}\begin{cases} x+3y_+-3y_-\le 10\\ 2x+y_+-y_-\le 15\\ x\ge 0\\ y_+\ge 0\\ y_-\ge 0\end{cases}$$

1.4 אלגוריתם simplex

המטרה העליונה של האלגוריתם היא למצוא את הפתרון האופטימלי אשר ממקסם את פונקציית המטרה מתוך אינסוף הפתרונות האפשריים אשר מוגדרים ע"י האילוצים. גיאומטרית, מרחב הפתרונות של אילוצי הLP יוצר צורה גאומטרית של רשת קודקודים. האלגוריתם לא סורק את כל השטח האינסופי הנ"ל, אלא "מטפס" מקודקוד לקודקוד על גבי המעטפת של האזור המותר של הפתרון, כך שבכל צעד הרווח רק גדל או נשאר זהה, עד שמגיע לקודקוד הגבוה ביותר בו נמצא המקסימום האבסולוטי.
פונקציית המטרה הינה:

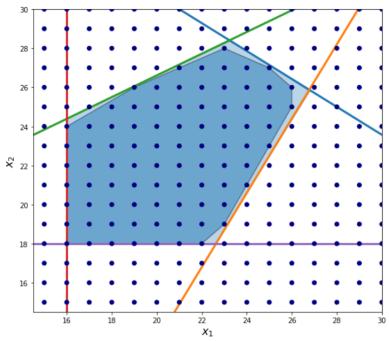
$$\max_{x=(x_1,\ldots,x_n)} c^T x$$

$$\forall i\in[m]: s_i=b_i-\sum_{j=1}^{n}a_{i,j}x_j\ \text{s.t}\ \forall i,j: x_i\ge 0, s_j\ge 0$$

באשר c מוגדר להיות וקטור המקדמים של המשתנים.

הגדרה 5. (Basic Feasible Solution). נקרא למשתנים $\{x_i\}_{i=1}^n$ משתנים לא בסיסיים (אלו של פונקציית המטרה), ויהיו $\{s_i\}_{i=1}^m$ משתני הבסיס של המערכת, שכן כל אחד מהם מבודד במשוואה משלו. מצב בו כל המשתנים הלא בסיסיים מאפסים את עצמם נקרא BFS (פתרון בסיסי פיזיבלי). אלגוריתם הSimplex מתחיל לרוץ במצב זה בו $x_1=x_2=\ldots=x_n=0$, במצב זה מתקיים כי לכל i: $s_i=b_i$ בתחילת הריצה.

הערה 6. נתבונן לצורך ההמחשה במערכת אילוצים על זוג משתנים x_1,x_2. האזור הצבוע בכחול הוא תחום ההצבה האפשרי למשתנים - כל נקודה בתוך המצולע הכחול מקיימת את האילוצים. הישרים - הם הגבולות שיוצרים את המצולע. הנקודות בתוך המצולע הכחול: אלו הנקודות של המשתנים בשלמים בתוך האזור של הפתרונות החוקיים.



1.4.1 דוגמה להרצת האלגוריתם

עוד לפני שנציג פסודו של האלגוריתם, נרצה לראות דוגמת הרצה עליו. נזכר בבעיית האופטימיזציה על השולחנות והכסאות שראינו בתחילת ההרצאה:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y\le 240\\ 2x+y\le 100\\ x,y\ge 0\end{cases}$$

נרצה להפוך אותה לבעיית אופטימיזציה מסוג שוויון, באמצעות משתני הSlack:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y+s_1=240\\ 2x+y+s_2=100\\ x,y,s_1,s_2\ge 0\end{cases}\implies \max\ (70x+50y)\ \text{s.t}\begin{cases}s_1=240-4x-3y\\ s_2=100-2x-y\\ x,y,s_1,s_2\ge 0\end{cases}$$

בשלב ראשון של האלגוריתם, מאפסים את המשתנים לפי BFS: $x=0,y=0\implies s_1=240,s_2=100$. במצב זה אנו למעשה לא מייצרים שום רהיטים, ומשתני העזר מקבלים את מלוא המשאבים הפנויים. כלומר נק' ההתחלה שנקראת BFS: Basic Feasible solution הוא פתרון בסיסי התחלתי בו כל המשתנים החופשיים הינם אפס בדיוק, ומשתני ההרפיה סופגים את כל המשאבים.

כעת, נרצה להגדיל את הרווח שלנו. שכרגע, הוא בדיוק אפס. נתבונן על המקדמים בפונקציית המטרה: המקדם של x הינו 70, גדול מהמקדם של y שהינו 50. לכן המסקנה הברורה: לx יש השפעה חיובית חזקה יותר על הרווח. לכן - הגיוני שהאלגוריתם ירצה להגדיל כמה שיותר את הערך של x. בשלב זה אנחנו נכניס "משתנה חדש למשחק" - פעולה זו נקראת Pivot. כעת נרצה לאפשר לx להפוך להיות משתנה פעיל. לפי המשוואה השניה ניתן לקבל:

$$x=50-\frac{y}{2}-\frac{s_2}{2}$$

כעת, נוכל להסתכל על הבעיה הבאה:

$$\max\ (70\cdot(50-\frac{y}{2}-\frac{s_2}{2})+50y)\ \text{s.t}\begin{cases}s_1=240-4\cdot(50-\frac{y}{2}-\frac{s_2}{2})-3y\\ x=50-\frac{y}{2}-\frac{s_2}{2}\\ x,y,s_1,s_2\ge 0\end{cases}$$

כלומר,

$$\max\ (3500+15y-35s_2)\ \text{s.t}\begin{cases}s_1=40-y+2s_2\\ x=50-\frac{y}{2}-\frac{s_2}{2}\\ x,y,s_1,s_2\ge 0\end{cases}$$

מה שינינו כאן בעצם? הרי, לכאורה ניתן לומר שסה"כ החלפנו את אותה משוואה במשוואה שקולה לה. ובכן, כמעט: אנחנו החלפנו את תפקידי המשתנים. בתחילה - x,y היו בחוץ (שווים לאפס), וs_1,s_2 היו בפנים - הם היו המשתנים שניהלו את העניינים. במצב זה, הרווח היה אפס. לאחר פעולת הPivot, קיבלנו משתנים לא בסיסיים חדשים: s_2,y.

שלב הבא: מאפסים את המשתנים הלא בסיסיים ומקבלים $s_2=y=0$. במצב זה ערך פונקציית המטרה הוא 3500. כלומר, מרווח של 0 עלינו לרווח של 3500 בצעד אחד בודד. בדומה, שוב נסתכל על המקדמים של המשתנים הלא בסיסיים. המקדם של y: 15 והמקדם של s_2: -35. כלומר, נדרשת איטרציה נוספת. כעת, נבודד את y כי המקדם שלו גבוה יותר וחיובי ממש. כלומר: יש דרך עוד לשפר אותו. לכן נבודדו ונקבל:

$$y=40+2s_2-s_1$$

ומכאן:

$$\max\ (3500+15\cdot(40+2s_2-s_1)-35s_2)\ \text{s.t}\begin{cases}y=40+2s_2-s_1\\ x=50-\frac{40+2s_2-s_1}{2}-\frac{s_2}{2}\\ x,y,s_1,s_2\ge 0\end{cases}$$

כלומר נקבל:

$$\max\ (4100-5s_2-15s_1)\ \text{s.t}\ \begin{cases} y=40+2s_2-s_1 \\ x=30+\frac{s_1}{2}+\frac{2s_2}{2} \\ x,y,s_1,s_2\geq 0 \end{cases}$$

בשלב זה, משתני ההרפיה הם x,y וכן משתני הבסיס הם s_1,s_2. כל המקדמים של משתני הבסיס שליליים, ולכן האלגוריתם מפסיק ריצתו. לפי BFS, נקבל כי $s_1=s_2=0$ ומהמשוואות נקבל כי הפתרון האופטימלי הינו:

$$x=30, y=40$$

הערה חשובה. נשים לב כי בכל שלב למשל בשלב של בידוד y היה לנו 2 משוואות שיכולנו לבודד מהם אותו. בשלב זה אנחנו מפעילים את **מבחן היחס המינימלי** אשר מטרתו להבטיח שכל משתני הבסיס ישארו אי שליליים. לדוגמה היה לנו $s_1=40-y+2s_2$. אם נציב $s_2=0$ (מדוע? השני במשתנים הלא בסיסיים) ונקבל $s_1=40-y$ לכן $y\leq 40$. כי $s_1\geq 0$. מהמשוואה השניה, אם נציב $s_2=0$ נקבל $x=50-\frac{y}{2}$. מכאן $y\leq 100$. לכן ־ נבחר את המשוואה הראשונה. בוחרים את החסם המחמיר יותר. אנחנו לוקחים את המחמיר מבניהם על מנת להבטיח ששאר משתני הבסיס ישארו אי שליליים.

1.4.2 מקרי קצה

עד כה הנחנו כי $b\geq 0$. כלומר, הוקטור החופשי ההתחלתי של האילוצים הוא חיובי או אפס. מה אם ההנחה הזו נכשלת ויש לנו לפחות ערך שלילי אחד בb? במצב כזה אי אפשר לאפס בכל שלב את המשתנים המקוריים $x=0$, כי אז נקבל כי $s_i=b_i<0$, כלומר משתני העזר יהפכו להיות שליליים וזה לא חוקי לפי כללי הLP. כלומר: אין לנו בכלל נקודת התחלה חוקית. כיצד פותרים את הבעיה הזו? לשם כך, נייצר בעיית עזר. על מנת למצוא קודקוד התחלתי חוקי שממנו נוכל להתחיל לרוץ עם סימפלקס, נבצע טריק מתמטי (שלב זה נקרא Phase 1).

- מכניסים משתנה חדש למערכת: x_0. מוסיפים את המשתנה המלאכותי לכל משוואות האילוצים, באגף ימין שלו עם סימן חיובי.
- משנים את פונקציית המטרה: במקום לנסות למקסם את הרווח המקורי שלנו, משנים את המטרה למטרה הבאה:

$$\max -x_0$$

כלומר, כיוון ש$x_0\geq 0$, הדרך למקסם את $-x_0$ היא לגרום לכך ש$x_0=0$.

$$\begin{array}{c} \max c^Tx \\ s_1=b_1-\sum_{i=1}^n a_{1,i}x_i \\ \vdots \\ s_m=b_m-\sum_{i=1}^n a_{m,i}x_i \end{array} \quad\Longrightarrow\quad \begin{array}{c} \max -x_0 \\ s_1=b_1+x_0-\sum_{i=1}^n a_{1,i}x_i \\ \vdots \\ s_m=b_m+x_0-\sum_{i=1}^n a_{m,i}x_i \end{array}$$

- מטרת הטריק: אם נצליח להריץ את אלגוריתם הסימפלקס ולהביא למצב שבו $x_0=0$, מצאנו פתרון בו כל האילוצים מתקיימים באופן חוקי ־ אנו מחזיקים בBFS חוקי שאפשר איתו לחזור לבעיה המקורית של מקסום הרווח ולרוץ עם האילוצים שלה. אם בסוף התהליך $x_0>0$ המשמעות היא שהבעיה המקורית כלל לא אפשרית ואין לה אף פתרון חוקי.
- כיצד מריצים זאת טכנית? על מנת להריץ את אלגוריתם הסימפלקס על בעיית העזר הנ"ל, אנחנו צריכים להכניס את x_0 לבסיס במקום השורה הבעייתית בה יש את הb השלילי ביותר. לכן, יהי

$$\ell=\arg\min_i(b_i)$$

כלומר, ℓ הוא האינדקס שבו הערך של b הוא הקטן ביותר ומכאן $b_\ell<0$. לוקחים את המשוואה של אותו אילוץ בעייתי בה הוא מופיע ומסדרים אותה מחדש על מנת לבודד את x_0 באופן הבא:

$$-x_0=b_\ell-s_\ell-\sum_{j=1}^n a_{\ell,j}x_j \Longrightarrow x_0=-b_\ell+s_\ell+\sum_{j=1}^n a_{\ell,j}x_j$$

מציבים את הביטוי הנ"ל לשאר המשוואות ולתוך פונקציית המטרה החדשה, ומפעילים את אלגוריתם הסימפלקס הרגיל על מנת למקסם את $-x_0$. נשים לב שכיוון ש$b_\ell < 0$ אזי $-b_\ell > 0$.

באשר האלגוריתם יעצור, אם נגיע למצב שבו הערך של x_0 ירד והתאפס לחלוטין מצאנו נקודה חוקית שבה כל משתני הs חיוביים - כעת ניתן להתעלם מx_0, לחזור לפונקציית המטרה המקורית עם האילוצים החדשים שהם BFS ולפתור את הבעיה האמיתית. כלומר:

$$\begin{gathered}\max b_\ell - s_\ell - \sum_{i=1}^{n} a_{\ell,i}x_i \\ s_1 = b_1 + x_0 - \sum_{i=1}^{n} a_{1,i}x_i \\ .. \\ x_0 = -b_\ell + s_\ell + \sum_{j=1}^{n} a_{\ell,j}x_j \\ s_m = b_m + x_0 - \sum_{i=1}^{n} a_{m,i}x_i\end{gathered}$$

כעת, אנחנו מחזיקים בבעיה שמשלימה את ההכנה של מערכת המשוואות ויש לנו פונקציית מטרה ומערכת אילוצים בה ניתן לאפס את כל המשתנים החופשיים $(x_1, ..., x_n, b_\ell)$. לאחר האיפוס שלהם, כל משתני האילוץ נוצרים חיוביים. שכן לאחר שנציב את x_0 בכל משוואות האילוצים נקבל:

$$\begin{gathered}\max b_\ell - s_\ell - \sum_{i=1}^{n} a_{\ell,i}x_i \\ s_1 = (b_1 - b_\ell) + s_\ell - \sum_{i=1}^{n}(a_{1,i} - a_{\ell,i})x_i \\ .. \\ x_0 = -b_\ell + s_\ell + \sum_{j=1}^{n} a_{\ell,j}x_j \\ s_m = (b_m - b_\ell) + s_\ell - \sum_{i=1}^{n}(a_{m,i} - a_{\ell,i})x_i\end{gathered}$$

נשים לב כי כעת לאחר שמציבים $x_1 = ... = x_n = b_\ell = 0$ מקבלים כי לכל $i \in [m]$ מתקיים $(b_i - b_\ell) \geq 0$ ויחד עם שאר האיברים, סה"כ נקבל כי המשתנים $s_i \geq 0$. כלומר, הצלחנו למנוע את המצב שבו לאחר הצבת פתרון הBFS המשתנים הופכים לשליליים.

על המערכת הנ"ל מריצים את אלגוריתם הסימפלקס עד שפונקציית המטרה מגיעה למקסימום שלה, בעת ש$x_0 = 0$ סיימנו את השלב הראשון ונגיע למערכת משוואות מהצורה הבאה:

$$\begin{gathered}y_1 = b'_1 - \sum_{i=1}^{n} a'_{1,i}z_i \\ y_2 = b'_2 - \sum_{i=1}^{n} a'_{2,i}z_i \\ .. \\ y_m = b'_m - \sum_{i=1}^{n} a'_{m,i}z_i \\ \forall i : z_i \geq 0, \forall j : y_j \geq 0\end{gathered}$$

הרי בסיום שלב העזר, האלגוריתם עוצר: על מנת לקרוא את הערכים של המשתנים בנק' הנוכחית שזהו הקודקוד שמצאנו, אנו מאפסים את כל המשתנים שיושבים בצד (כאן אלו $z_1, ..., z_n$). לשאר משתני המערכת - נקבל לכל i: $y_i = b'_i$. כלומר, בסיום נקבל כי $y = b'$ הוא וקטור ערכים המקיים $b' \geq 0$. לכן מצאנו קודקוד התחלתי חוקי ואמיתי ממנו ניתן להתחיל לפתור את האופטימיזציה של הבעיה האמיתית.

מסקנה 7. ניתן להסיר את x_0 ולקבל שישנו פתרון פיזיבלי למערכת אם ורק אם בסיום התהליך $x_0 = 0$. מדוע? הוא הרי לא היה משתנה אמיתי מלכתכילה של הבעיה שלנו. הוא היה שתול מלאכותי שנועד על מנת לחלץ אותנו ולמצוא לנו נק' התחלה חוקית. אם הוא לא הגיע לאפס בסיום, משמעות הדבר שאין פתרון חוקי. כלומר, אם הרצנו את הסימפלקס על המערכת המלאכותית שבנינו וגילינו בסיום כי בהכרח $x_0 > 0$ אזי אין שום שילוב בעולם של מס' אי שליליים שיכולים לספק את מערכת המשוואות. האילוצים פשוט סותרים זה את זה - למשל: אילוץ שדורש שהסכום לא יעבור את 50 ואילוץ אחר שדורש שיהיה לפחות 51. לכן - הבעיה *infeasible*.

בסיום, נקבל את בעיית האופטימיזציה הבאה:

$$\begin{gathered}\max \ (c')^T z \text{ s.t } z = (z_1, ..., z_n) \\ y_1 = b'_1 - \sum_{i=1}^{n} a'_{1,i}z_i \\ y_2 = b'_2 - \sum_{i=1}^{n} a'_{2,i}z_i \\ .. \\ y_m = b'_m - \sum_{i=1}^{n} a'_{m,i}z_i \\ \forall i : z_i \geq 0, \forall j : y_j \geq 0\end{gathered}$$

וכן $c'_i = c_i + \sum_{j=1}^{m} y_j a'_{j,i}$ (כלומר אלו ערכי המקדמים החדשים בפונקציית המטרה).

1.4.3 פסאודו קוד

להלן תיאור של האלגוריתם בפסאודו קוד:

Algorithm: Simplex with Phase 1 & Phase 2

Input: Matrix A, vectors b and c
Phase 1: Feasibility

1. Find the most negative b_i (let's call it b_ℓ)
2. If $b_\ell \geq 0$: already feasible! Skip to Phase 2.
3. Else:
   (a) Add artificial variable x_0 and set objective to Max: $-x_0$
   (b) Pivot on the b_ℓ row to make x_0 basic
   (c) Run Simplex until $\max(-x_0)$ is reached
   (d) If optimal $x_0 > 0$: return "Infeasible" (no solution)
   (e) If optimal $x_0 = 0$:
      i. Remove x_0 from the system
      ii. We now have a valid initial BFS ($b' \geq 0$)

Phase 2: Optimality

1. Restore the original objective function (expressed with current non-basic variables z, yielding c')
2. While there is any non-basic variable with a positive coefficient in c':
   (a) Choose an entering variable (positive coefficient in c')
   (b) Perform the Minimum Ratio Test to choose the leaving variable
   (c) Perform Pivot
   (d) If no variable can leave (all coefficients ≤ 0 for the entering variable): return "Unbounded"
3. Return the optimal solution

האלגוריתם לעיל מתאר באופן פורמלי את התהליך שתיארנו. ביתר פירוט, בשלב ב' אנו מחזיקים בפתרון BFS ומעוניינים לשפר אותו. למעשה השיפוע הוא המקדם בc'. לכן, בוחרים מקדם חיובי c' (כלל בחירה הגיוני הוא המקסימלי בשיפועו) ומתקדמים.
שלב שבו נרצה להתעכב הוא הRatio Test: בעת שמצאנו משתנה עם שיפוע חיובי, נרצה לסדר את המערכת. נזכר בדוגמה שראינו בתחילה, שם בשלב הראשון בחרנו לסדר את המשוואה על x ולקבל:

$$x = 50 - \frac{y}{2} - \frac{s_2}{2}$$

כלומר הגדלנו אותו ל50, זהו בדיוק השלב שבו מתבצע הRatio Test. הרי, יכולנו לסדר את המשוואה בצורה אחרת ולהגדילו לפי המשוואה הראשונה (שם היינו מגדילים אותו ל60). מדוע לא עשינו זאת? תמיד נבחר במשוואה של משתנה הלא־בסיס שמגדילה כמה שבפחות. שכן, כל הפתרונות האחרים אינם חוקיים. הרי, כדי להשאר חוקי בכל האילוצים בו זמנית מוכרחים לעצור בתקרה הנמוכה ביותר מבניהן.
ביצוע הPivot הוא השלב בו x מוצב לתוך בעיית האופטמיזציה החדשה והופך למשתנה בסיס, ומקבלים בעיית אופטימיזציה חדשה.

1.4.4 זמן ריצה

נשים לב כי כל הפעולות האלגבריות בסימפלקס הינן פעולות אלמנטריות על שורות, ערך פונקציית המטרה יישאר עקבי ושקול לאחר הפעולות ומשוואת המטרה תמיד תובע במונחי המשתנים הלא בסיסיים בלבד. ברגע שבוחרים קבוצה קבועה של משתנים לא בסיסיים, מקבלים פתרון בסיסי אפשרי יחיד ומוגדר היטב.

למרות שבתאוריה, הסיבוכיות גבוהה מאוד, בפועל האלגוריתם רץ מהר מאוד על בעיות אמיתיות. כמו כן, ישנן מס' היורסטיקות שונות לבחירת Pivot כדי לשפר את מהירות זמן הריצה. חייבים להזהר ולא להכנס ללולאות אינסופיות - לכן ישנו כלל בלנד עליו לא הרחבנו בקורס.

משפט 8. זמן הריצה של אלגוריתם *Simplex* הוא:

$$\mathcal{O}\left(\binom{n+m}{n}\cdot nm\right)$$

באשר n הוא מס' המשתנים הלא בסיסיים, וכן m הוא מס' האילוצים. במערכת סה"כ יש $n+m$ משתנים.

- בחלק הראשון: למעשה אלגוריתם הסימפלקס כל האלגוריתם עובר מקודקוד לקודקוד בתוך פאון גאומטרי. קשה לחסום את זה - אבל ישנו חסם תאורטי מתמטי למספר הפעמים שזה יקרה. הרי מה קורה בפועל? בכל פעם בוחרים בסיס. ברגע שיש בסיס שמשתמשים בו, הוא לא ישוב שוב. לכן ניתן לומר שבחירת הבסיס האופטימלי שקולה לבחירת n משתנים מתוך $n+m$ משתנים - כך שn הנבחרים יהיו לא בסיסיים. מס' הדרכים לעשות זאת כמובן $\binom{n+m}{n}$.
- בחלק השני: כל בחירת בסיס כזה מביעה צעד Pivot. מה כולל צעד פיבוט בודד? שלושה שלבים.

1. בחירת המשתנה הנכנס - בדיקת פונקציית המטרה ולקיחת המשתנה בעל השיפוע הגדול ביותר.
2. בחירת המשתנה היוצא - בודקים את האילוצים באמצעות מבחן היחס המינימלי כדי לזהות איזה משתנה בסיס קיים יגיע לאפס ראשון ויחסום את הגדלתו של המשתנה הנכנס.
3. עדכון אלגברי של הטבלה: זה החלק שלוקח רוב הזמן. יש לעדכן את המטריצה בגודל $n\times m$ כך שאנחנו מחליפים תפקידים: כלומר פיבוט זו ההצבה שאנחנו עושים למשתנה הבסיס שנבחר על מנת שיכנס בכל המקומות כמשתנה לא בסיסי.

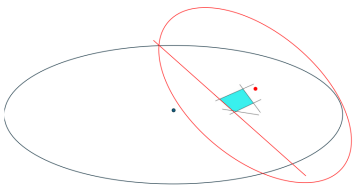
לכן סה"כ שלב זה עולה $\mathcal{O}(n\cdot m)$.

הערה 9. נשים לב שהסימפלקס לא פשוט "עובר על כל האפשרויות" בצורה נאיבית, אלא מתקדם מפתרון פיזיבלי עד שיגיע לפתרון שבו כל המקדמים אי שליליים. עם זאת, במקרה הגרוע ביותר הוא אכן יתחיל מנקודה גרועה מאוד גאומטרית ויצטרך לעבור על כל האפשרויות לנקודה חוקית עד שיגיע לנקודה נכונה. לכן החסם העליון לזמן הריצה אקספוננציאלי, למרות שמדובר באלגוריתם חמדן שרץ בזמן ריצה טוב למדי.

1.5 אלגוריתם הEllipsoid

בעוד ראינו כי אלגוריתם הסימפלקס הינו אקספוננציאלי, האלגוריתם שנציג הוא פולינומי: וזה ההבדל הגדול בניהם. עד להופעת האלגוריתם, לא היה ידוע אם תכנון לינארי היא בעיה שנמצאת בP. בעת שהגיע האלגוריתם לעולם, הספק אודות השייכות של תכנון לינארי לP נעלם.

ובכן, כיצד קשורה האליפסה? האלגוריתם מתחיל לסרוק את המרחב עם אליפסה ענקית - שבוודאות מכילה בתוכה את כל אזור הפתרונות האפשריים. בתוך האליפסה - צורה גאומטרית קטנה שמכילה את אזור הפתרונות האפשריים לבעיה. בכל איטרציה: האלגוריתם בודק האם מרכז האליפסה נמצא בתוך אזור הפתרונות. אם כן: מצאנו פתרון. אם לא: האלגוריתם מוצא אילוץ מופר שעובר דרך המרכז, ובאמצעותו בונה אליפסה חדשה וקטנה יותר (בהמחשה - זו באדום) שמקיפה רק את החצי של האליפסה הקודמת שבו עדיין עשוי להמצא אזור הפתרונות. בכל צעד האלגוריתם מכווץ את נפח האליפסה בקצב אקספוננציאלי קבוע, עד שננעל על אזור הפתרונות או שמוכיח שהוא ריק.



אלגוריתם האליפסואיד לא יותר לפתור בעיית אופטימיזציה - כלומר למצוא את המקסימום של c^Tx. לכן מבצעים טריק מתמטי: ממירים את בעיית האופטימיזציה לבעיית החלטה בעזרת חיפוש בינארי:

- נתונה בעיית אופטימיזציה $\max\{c^Tx|Ax\leq b, x\geq 0\}$.
- אנו מנחשים ערך מטרה כלשהו M ומנסים לענות על השאלה: האם קיים פתרון כך ש$c^Tx\leq M$? יחד עם שאר האילוצים.
- אם האלגוריתם מוצא פתרון, סימן שניתן לשאוף גבוה יותר. לכן ננסה ערך M גבוה יותר. אם האזור ריק, ננסה ערך נמוך יותר של M. כך מריצים חיפוש בינארי על ערך האופטימום M עד שמתכנסים לאופטימום המדויק.

משפט 10. אם קיים פתרון אפשרי למערכת האילוצים $Ax \leq b$ ו־$x \geq 0$ אזי תמיד קיים לפחות פתרון אחד שבו כל הערכים קטנים או שווים לחזקה אקספוננציאלית:

$$\forall i : x_i \leq n^{5n}$$

מה המשמעות של המשפט הנ"ל? כדי שאלגוריתם גאומטרי כמו אליפסואיד יעבוד, אנו חייבים לדעת כי אזור הפתרונות לא "בורח" לאנסוף או שקטן מדי כדי שנוכל לתפוס אותו. המשפט הנ"ל מבטיח כי אם קיימת נקודה אפשרית כלשהי, בהכרח קיימת לפחות נקודה אחת שנמצאת בתוך קוביה סופית וחסומה היטב בגודל n^{5n}. כלומר - אנחנו יכולים להתחיל את האלגוריתם עם אליפסה ראשונית ענקית שמכילה קוביה זו, ויודעים בוודאות כי אם יש פתרון: הוא שם בפנים. נעיר כי $x \in \mathbb{R}^n$, לכן הכוונה שכל אחד מהערכים בפתרון $x = (x_1, ..., x_n)$ מקיים את החסם.

משפט 11. המערכת המקורית $Ax \leq b, x \geq 0$ היא בעלת פתרון אם ורק אם המערכת $Ax \leq b + \varepsilon \ s.t \ \varepsilon = \frac{1}{n^{5n}}$ היא בעלת פתרון.

מדוע אנו זקוקים למשפט זה עבור נכונות האלגוריתם? במחשבים, מבחינה פרקטית, לא ניתן לייצג מס' ממשיים באינסוף דיוק, לכן אי אפשר לבצע בדיקות אבסולוטיות של "האם הנקודה נמצאת על הישר בדיוק". המשפט הנ"ל מבטיח כי אם נרחיב מעט את האילוצים באופן זעיר, לא נשנה את העובדה שקיים פתרון. כלומר, אם היא פתירה היא תשאר פתירה גם עם ההרחבה הקטנה הזו.

1.6 Integer Programming vs. Linear Programming

הגדרה 12. בעיה מסוג Integer Programming היא בעיה שמנוסחת באופן דומה לLP, בתוספת הדרישה כי המשתנים הם מספרים שלמים. כלומר, לכל i מתקיים $x_i \in \mathbb{Z}$.

משפט 13. IP (*Integer Programming*) היא בעיה NP־קשה.

כלומר, ישנו הבדל עצום בין מציאת פתרון בשלמים למציאת פתרון כלשהו. בניגוד לפתרון כלשהו, שניתן לפתרון פולינומי באמצעות אליפסואיד, לא ידוע כיום על אלגוריתם פולינומי שפותר את IP וכנראה שלא קיים כזה.

1.7 כיסוי קודקודים (Vertex Cover)

הגדרה 14. יהי $G = (V, E)$ גרף. קבוצת קודקודים $S \subseteq V$ נקראת כיסוי קודקודים אם לכל $(u, v) \in E$ מתקיים $u \in S$ או $v \in S$.

כפי שראינו בחישוביות וסיבוכיות, מדובר בבעיה NP־שלמה. ניתן לייצג אותה באופן מתמטי כבעיית IP. בהינתן כיסוי $S \subseteq V$ לכל $v \in V$ נגדיר משתנה בינארי:

$$x_v = \begin{cases} 1 & v \in S \\ 0 & v \notin S \end{cases} \in \{0, 1\}$$

נקבל את בעיית האופטימיזציה הבאה:

$$\min \sum_{v \in V} x_v \text{ s.t } \forall (u, v) \in E : x_u + x_v \geq 1$$

כלומר, IP לפחות קשה כמו VC שהיא הרי NPH. מכאן נקבל כמסקנה שאכן $IP \in NPH$.

1.8 קליקה (Clique)

הגדרה 15. יהי $G=(V,E)$ גרף. קבוצת קודקודים $S\subseteq V$ נקראת קליקה אם לכל $u,v\in S$ מתקיים $(u,v)\in E$.

תהי S תת קבוצה של קודקודים. נגדיר:

$$x_v=\begin{cases}1 & v\in S\\ 0 & v\notin S\end{cases}\quad\in\{0,1\}$$

נקבל את בעיית האופטימיזציה הבאה:

$$\max\sum_{v\in V}x_v\ \text{s.t}\ \forall(u,v)\notin E: x_u+x_v\leq 1$$

מדוע האילוץ נראה ככה? לפי הגדרת קליקה, כל זו בקבוצה חייב להיות מחובר בקשת. לכן, אם נקח זוג צמתים שאין בניהם קשת בגרף אסור לשניהם להבחר בו זמנית לקליקה. אם אחד מהם יהיה 1, האחר חייב להיות אפס. נשים לב שבאמצעות אילוץ זה אנו אוכפים קליקה. מתי קליקה יכולה להיות בעייתית? אם בחרנו זוג קודקודים שאין ביניהם קשת - זו בהכרח לא קליקה. בכל מקרה אחר, כל הקודקודים שנבחרו לקבוצה יש בינם לבין עצמם קשת לכל הזוגות. לכן, נדרוש מכל הזוגות של הקודקודים שאין בניהם קשת, שרק אחד לכל היותר יכנס.

1.9 Weak Duality

כאשר אנחנו פותרים בעיית מקסימיזציה, לפעמים קשה למצוא את הפתרון המדויק או להוכיח מהו החסם העליון שלו. הרעיון של דואליות חלשה, הוא למצוא חסם עליון לערך המקסימלי של פונקציית המטרה ע"י שילוב לינארי של האילוצים הנתונים. לצורך כך, נפתח בדוגמה

1.9.1 דוגמה

נתבונן בבעיית האופטימיזציה הבאה:

$$\max(3x_1+2x_2)\ \text{s.t}\begin{cases}2x_1-x_2\leq 10\\ 6x_2-x_1\leq 5\\ x_1,x_2\geq 0\end{cases}$$

נסיון ראשון. נקח את זוג האילוצים. נראה כי:
אם נכפול ב3 את האילוץ הראשון וב2 את האילוץ השני נקבל:

$$3x_1+2x_2\leq 4x_1+10x_2=3\cdot(2x_1-x_2)+2(6x_2-x_1)\leq 3\cdot 10+2\cdot 5=40$$

כלומר, ישנו חסם של 40 לערך פונקציית המטרה.
נסיון שני, חכם יותר. נכפול ב2 את האילוץ הראשון וב1 את האילוץ השני ונקבל:

$$3x_1+2x_2\leq 3x_1+4x_2=2\cdot(2x_1-x_2)+(6x_2-x_1)\leq 2\cdot 10+5=25$$

כלומר, חסמנו את ערך פונקציית המטרה כעת ב25. חסם יותר טוב.
ובכן, במקום לחפש ידנית מספרים ע"י ניחוש - למה שלא נחפש את השילוב האופטימלי שיתן לנו את החסם העליון ההדוק ביותר? יהיו y_1,y_2 משתנים שיחליפו את הערכים הקבועים. נראה כי הצ"ל המשוקלל הינו:

$$y_1\cdot(2x_1-x_2)+y_2\cdot(6x_2-x_1)\leq 10y_1+5y_2\implies$$

$$(2y_1-y_2)x_1+(-y_1+6y_2)x_2\leq 10y_1+5y_2$$

נרצה שיתקיים

$$3x_1 + 2x_2 \leq (2y_1 - y_2)x_1 + (-y_1 + 6y_2)x_2 \leq 10y_1 + 5y_2$$

לכן נדרוש כי $2y_1 - y_2 \geq 3, -y_1 + 6y_2 \geq 2$ (זהו חסם על המקדמים). מכאן, נקבל בעיית אופטימיזציה למציאת אבי־אבות החסמים העליונים. כלומר:

$$\min(10y_1 + 5y_2) \text{ s.t} \left\{ \begin{array}{c} -y_1 + 6y_2 \geq 2 \\ 2y_1 - y_2 \geq 3 \\ y_1, y_2 \geq 0 \end{array} \right\}$$

זוהי יצירתו של הDual, מתוך הPrimal. בעיית מקסימיזציה של בחירת משאבים, הופכת לבעיית מינימום של תמחור עבור אותם משאבים. כך למעשה, נוכל למצוא חסם עליון (קטן ביותר, הדוק ביותר - לכן זו בעיית מינימיזציה) לערך פונקציית המטרה של בעיית המקסימיזציה.

1.9.2 הגדרה פורמלית

בהינתן Primal, בעיית מקסימיזציה כללית עם n משתנים וm אילוצים, נרצה למקסם את

$$\max \ c_1x_1 + ... + c_nx_n \text{ s.t} \left(\begin{array}{ccccc} a_{1,1}x_1 + & a_{1,2}x_2 & +.. & + & a_{1,n}x_n \leq b_1 \\ & & & & \\ & & & & \\ a_{m,1}x_1 + & a_{m,2}x_2 & +.. & + & a_{m,n}x_n \leq b_m \end{array} \right)$$

ובאופן שקול:

$$\max c^T x \text{ s.t } Ax \leq b$$

יהיו $y_1, ..., y_m$ משקולות. המטרה שלנו היא למזער את סכום האילוצים כפול המשקולות. כלומר:

$$\min \sum_{j=1}^{m} b_j y_j \equiv \min \ b^T y$$

הרעיון הוא כי לכל אילוץ j בוחרים משקולת $y_j \geq 0$, כופלים בה את האילוץ ומחברים את כל האילוצים. סה"כ מקבלים

$$b_1y_1 + ... + b_my_m = b^T y$$

כלומר - זה מה שיש באגף ימין: מכפלה סקלרית של b החסם ככפל במשקולות. באגף שמאל, לאחר החיבור המקדם של כל משתנה x_i הוא סכום המקדמים שלו בכל האילוצים, כל אחד כפול המשקולת של האילוץ שלו:

$$x_i = a_{1,i}y_1 + ... + a_{m,i}y_m = \sum_{j=1}^{m} a_{j,i}y_j$$

כלומר, סכימת השורה הi של A מוכפלת ב$\{y_i\}_{i=1}^m$. אנו דורשים כי המקדם של כל משתנה בצירוף יהיה לפחות המקדם בפונקציית המטרה. כלומר:

$$\forall i \in [n] : c_i \leq \sum_{j=1}^{m} a_{j,i}y_j$$

סה"כ בכתיב מטריציוני - $A^T y \geq c$.
סה"כ הבעיה הדואלית תוגדר להיות:

$$\min \sum_{j=1}^{m} b_j y_j \ \forall i \in [n] : \sum_{j=1}^{m} a_{j,i} y_j \geq c_i \ , y_1, ..., y_m \geq 0$$

או באופן שקול:

$$\min b^T y \text{ s.t } A^T y \geq c, y \geq 0$$

משפט 16. (משפט הדואליות החלשה) כל פתרון דואלי, בהכרח חוסם את פתרון ה$Primal$.
הוכחה: מתקיים -

$$b^T y = y^T b \geq_* y^T A x =_{**} (A^T y)^T x \geq c^T x$$

שכן $*$ נכון כי $b \geq Ax$ וכן $**$ נכון כי מהאילוצים הדואלים דרשנו כי יתקיים $\forall i \in [n] : \sum_{j=1}^{m} a_{j,i} y_j \geq c_i$ ובאופן שקול $A^T y \geq c$.
לכן סה"כ

$$b^T y \geq c^T x$$

ולכן, כל ערך של הפתרון הדואלי חוסם את ערך הפתרון בפרימיאל.

■

0<0